

SEMINARIO DE HERRAMIENTAS MDE

StarUML: Modelado Semántico UML y Automatización Directa/Inversa

Análisis técnico de la herramienta en el desarrollo de software dirigido por modelos (MDD) y su formato extensible JSON

ASIGNATURA

Desarrollo de Software Dirigido por Modelos (DSM)

ETS de Ingeniería Informática (ETSINF) • UPV

📅 29 de mayo de 2026

EQUIPO DE EXPOSICIÓN:

👤 Jorge Cleigues Malet (Ponente 1)

👤 Nahuel Chosco Flores (Ponente 2)

👤 Brad Owen Valladares Quispe (Ponente 3)

👤 Jordi Barrachina Méndez (Ponente 4)

👤 Sergio Camacho Roig (Ponente 5)

Resumen e Información Técnica

Resumen del Propósito

StarUML es un modelador semántico de software diseñado para construir y manipular modelos visuales estructurados bajo los estándares UML 2.x, SysML y ERD. Actúa como núcleo en flujos MDE al separar la definición semántica del diseño y habilitar la ingeniería de código.



Entidad Comercial: Desarrollado y comercializado por **MKLab**.



Tecnología: Multiplataforma (basado en Electron/Node.js/HTML5) disponible para Windows, macOS y Linux.

Ficha Técnica Comercial

Sitio Web Oficial staruml.io 

Tipo de Licencia **Comercial / Evaluación Gratuita**

Restricciones Evaluación **Ilimitada en tiempo (Marca de agua ocasional)**

Precio Licencia Personal **\$99 USD (Pago único)**

Ecosistema Extensiones **Abierto y comunitario (GitHub)**

Descripción: Modelado Semántico vs. Dibujo

En el Desarrollo Dirigido por Modelos, un modelo es un elemento semántico con lógica de negocio y reglas bien definidas, no una simple ilustración vectorial.



El Modelo (Semántico): Es la base de datos de elementos estructurada jerárquicamente en memoria. Guarda la lógica (ej: la relación de herencia entre Cliente y Persona).



El Diagrama (Visualización): Es una vista bidimensional de una porción del modelo. Si borras un elemento del *diagrama*, este sigue existiendo en el *modelo*.



Validación Asíncrona: StarUML valida la sintaxis del modelo en segundo plano (ej: impide que una clase se herede a sí misma), asegurando la integridad lógica antes de generar código.



Modelos y Diagramas: Soporta diagramas estructurales y de comportamiento de UML 2.x (como diagramas de Clases, Estados, Secuencia, Actividades y Casos de Uso), integrándolos bajo una misma raíz lógica en el modelo.

Arquitectura del Elemento

El Modelo (AST)

```
Class: Usuario
└ Attr: password
└ Operation: login()
└ Parent: Persona
```

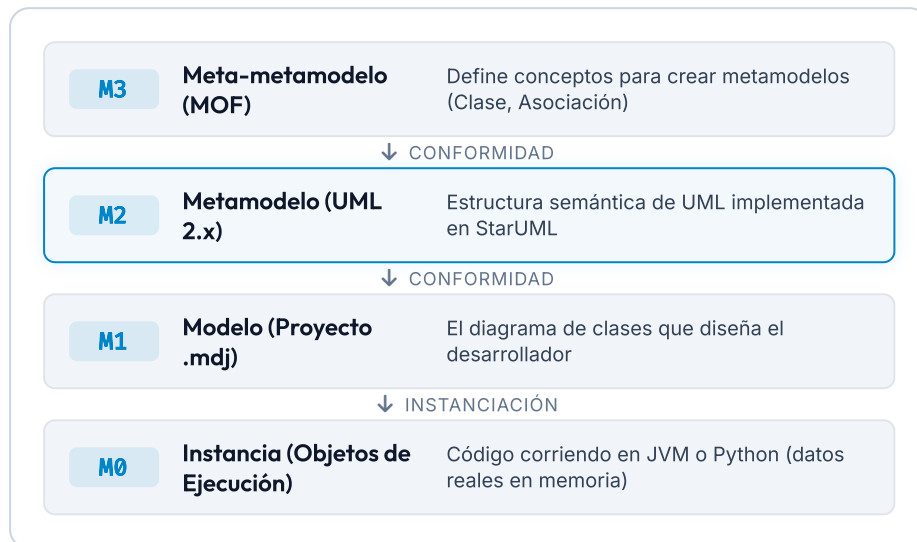
El Diagrama (Vista)

Usuario



Persona

Relación con Conceptos: La Jerarquía MOF



StarUML se ubica estrictamente dentro del marco de ingeniería dirigido por modelos promovido por la ****OMG (Object Management Group)****.

Soporte Metamodelo M2: StarUML define de forma estricta sus menús y lógica en función del estándar UML a nivel M2. Esto permite que el modelo M1 creado respete la semántica teórica de la disciplina.

Conformidad Estricta: Cada nodo dibujado a nivel M1 (instancia de modelo) conforma de manera directa a la definición formal de los elementos descritos a nivel M2.

El Formato del Modelo: El Archivo .mdj

Frente al estándar formal XML de intercambio (XMI) de la OMG (habitualmente complejo y pesado), StarUML utiliza una alternativa de almacenamiento moderna basada en texto plano: ****JSON**** con extensión **.mdj**.



Sintaxis Legible: Almacenado como un árbol JSON anidado nativo, lo que permite su inspección directa mediante cualquier editor de texto o scripts.



UUIDs para Trazabilidad: Cada elemento del modelo cuenta con una propiedad única `\_id` alfanumérica, permitiendo referencias cruzadas y relaciones inequívocas.



Facilidad de Parsing (MDE): Facilita enormemente escribir traductores personalizados con pocas líneas de código en Node.js o Python sin requerir SDKs complejos de modelado.

modelo.mdj

JSON

```
{
  "_type": "UMLModel",
  "_id": "AAAAAAFF+h6SjaM",
  "name": "ModelosClase",
  "ownedElements": [
    {
      "_type": "UMLClass",
      "_id": "UMLClass_Persona",
      "name": "Persona",
      "attributes": [
        {
          "_type": "UMLAttribute",
          "name": "nombre",
          "type": "String"
        }
      ]
    }
  ]
}
```

Transformación de Modelos: Directa (M2T)

La ingeniería directa o generación de código (Model-to-Code) traduce la estructura lógica de los modelos en ficheros de texto plano ejecutables.



Generación Basada en Plantillas (M2T): El motor lee el AST (árbol de sintaxis) en JSON del archivo .mdj, y mediante un motor de plantillas (EJS/Mustache) genera el código de clases.



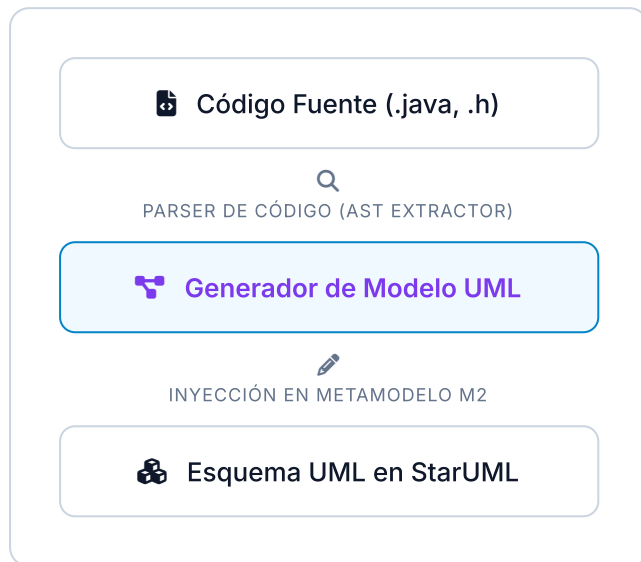
Determinismo: A diferencia de la IA Generativa, la transformación de modelos clásica en MDD es **100% determinista y reproducible**; dadas las mismas entradas y transformaciones, la salida es idéntica.



Lenguajes Soportados: Java, Python, C++, C#, PHP y JavaScript mediante plugins integrados en la aplicación.



Ingeniería Inversa y Sincronización (T2M)

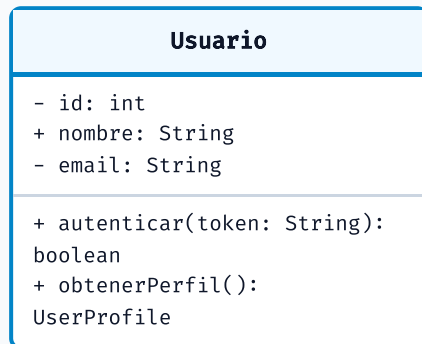


La ingeniería inversa o transformación de Texto a Modelo (T2M) permite reconstruir modelos visuales a partir de código escrito a mano.

-  **Extracción de AST:** Los plugins analizan la sintaxis del código de entrada (ej: clases Java), identifican tipos de variables, modificadores de acceso, herencias y firmas de métodos, mapeándolos en el JSON del modelo.
-  **Desafío de la Vista Visual:** La reingeniería recupera la semántica del modelo, pero no las coordenadas visuales del diagrama, obligando al usuario a distribuir las cajas manualmente en el lienzo.
-  **Round-trip Engineering:** La sincronización bidireccional continua (alineación de cambios de código a modelo y viceversa de forma transparente) es parcial en StarUML y requiere cuidado manual.

DEMO: Modelado y Generación de Código en Vivo

Vista StarUML (Canvas)



```
// Generado automáticamente por StarUML M2T (Determinista)
public class Usuario {

    private int id;
    public String nombre;
    private String email;

    public Usuario() {
        // Constructor
    }

    public boolean autenticar(String token) {
        // TODO: Retorno de boolean
        return false;
    }

    public UserProfile obtenerPerfil() {
        // TODO: Retorno de UserProfile
        return null;
    }

}
```


Comparativa Técnica de Herramientas MDE

Herramienta	Rigor MDD	Formatos de Intercambio	Extensibilidad	Curva e Interfaz
StarUML	ALTO (Fuerte validación UML2)	JSON (.mdj) nativo / XML XMI por Plugin	EXCELENTE (API web Javascript estándar)	Muy intuitiva, rendimiento ágil
Eclipse Papyrus	MÁXIMO (Estándar formal UML2/OMG)	XMI Nativo de Eclipse (EMF/Ecore)	COMPLEJO (Plugins Eclipse, Java, OSGi)	Muy compleja, rendimiento pesado
Enterprise Architect	MUY ALTO (Simulaciones y perfiles)	XMI de OMG estándar, bases de datos	MEDIO (APIs de automatización COM/.NET)	Saturada, curva muy pronunciada
Visual Paradigm	ALTO (Ingeniería de código y BD)	XMI de OMG, XML propietario	MEDIO (Plugins Java pesados)	Curva media, interfaz sobrecargada
Draw.io / Lucidchart	NULO (Solo dibujo vectorial básico)	XML vectorial sin semántica de modelo	INEXISTENTE (No interpreta metamodelos)	Extremadamente sencilla y rápida

VALORACIÓN CRÍTICA Y CONCLUSIONES

Evaluación Académica de StarUML

Un balance de la experiencia de uso de la herramienta contrastada con la teoría de la asignatura.



Valoración e Instalación

Instalación sencilla (Electron) y curva de aprendizaje muy baja. Ofrece una usabilidad y agilidad excelentes para modelado UML rápido frente a Eclipse Papyrus.



Rol de la Asignatura

La teoría de DSM (niveles MOF, metamodelos M2 y árboles AST) ha sido clave para comprender la estructura semántica de la herramienta y sus capacidades de automatización.



Limitaciones Críticas

Carece de validación OCL nativa y transformaciones M2M (ATL/QVT) directas en el flujo. La persistencia en JSON propietario debilita la compatibilidad con el estándar XMI de OMG.



Recursos y Fuentes

Consultamos la documentación oficial (docs.staruml.io), la especificación UML 2.5 de la OMG, y repositorios de extensiones comunitarias en GitHub.

¿Preguntas o comentarios sobre el Seminario?

¡Muchas gracias por su atención de parte del equipo!